YZ 103 · INTRODUCTION TO PROGRAMMING

Week 4 Lab

Decisions in Java

DELIVERED BY:	Teaching Assistants
FOLLOWS:	Lecture — Making Decisions in Java
FORMAT:	Write the program, compile it, test every sample run

GROUND RULES

- No flowcharts to draw this week: each page gives a task, a complete Java solution and real sample runs. Cover the solution while you work. Only Q13 is given as a drawing.
- Questions get harder from Part A to Part E. Part C introduces logical operators; Part D switch and ?: ; Part E combines everything.
- Test every sample run, then invent a run of your own that tries a boundary: the exact edge of each condition (18, 90, 100, 0 …).
- Every solution was compiled and run; the outputs shown are the real program output.

Q1. Read an integer and print whether it is positive, negative or zero.

JAVA SOLUTION

Q1.java

```java
import java.util.Scanner;

public class Q1 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int n = in.nextInt();
    if (n > 0) {
      System.out.println("positive");
    } else if (n < 0) {
      System.out.println("negative");
    } else {
      System.out.println("zero");
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 5
positive
```

RUN 2

```
> -3
negative
```

RUN 3

```
> 0
zero
```

HINT

A three-way choice is an if / else if / else ladder. Zero is neither positive nor negative — it needs its own branch.

Q2. Read an integer. Print "even" or "odd"; then, as a separate check, also print "divisible by 5" when it is.

JAVA SOLUTION

Q2.java

```java
import java.util.Scanner;

public class Q2 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int n = in.nextInt();
    if (n % 2 == 0) {
      System.out.println("even");
    } else {
      System.out.println("odd");
    }
    if (n % 5 == 0) {
      System.out.println("divisible by 5");
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 10
even
divisible by 5
```

RUN 2

```
> 7
odd
```

RUN 3

```
> 25
odd
divisible by 5
```

HINT

These are two independent questions, so use an if-else followed by a second, separate if — not a single ladder.

Q3. Read an age. If it is 18 or more print "You can vote."; otherwise print how many years are left until 18.

JAVA SOLUTION

Q3.java

```java
import java.util.Scanner;

public class Q3 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int age = in.nextInt();
    if (age >= 18) {
      System.out.println("You can vote.");
    } else {
      System.out.println("You can vote in " + (18 - age) + " years.");
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 20
You can vote.
```

RUN 2

```
> 15
You can vote in 3 years.
```

RUN 3

```
> 18
You can vote.
```

HINT

The number of years left is 18 - age. Check the boundary: age 18 can vote.

Q4. Read a score from 0 to 100 and print its letter grade (A 90+, B 80+, C 70+, D 60+, otherwise F). Anything outside 0–100 prints "Invalid score".

JAVA SOLUTION

Q4.java

```java
import java.util.Scanner;

public class Q4 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int score = in.nextInt();
    if (score < 0 || score > 100) {
      System.out.println("Invalid score");
    } else if (score >= 90) {
      System.out.println("A");
    } else if (score >= 80) {
      System.out.println("B");
    } else if (score >= 70) {
      System.out.println("C");
    } else if (score >= 60) {
      System.out.println("D");
    } else {
      System.out.println("F");
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 95
A
```

RUN 2

```
> 85
B
```

RUN 3

```
> 59
F
```

RUN 4

```
> 105
Invalid score
```

HINT

Test the invalid range first, with ||. After that, each test only needs a lower bound, because the earlier tests already ruled out higher scores.

Q5. Read an age and print the ticket price: under 6 is free, 6–17 pays 10 TL, 18–64 pays 25 TL, 65 and over pays 15 TL.

JAVA SOLUTION

Q5.java

```java
import java.util.Scanner;

public class Q5 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int age = in.nextInt();
    int price;
    if (age < 6) {
      price = 0;
    } else if (age < 18) {
      price = 10;
    } else if (age < 65) {
      price = 25;
    } else {
      price = 15;
    }
    System.out.println("Price: " + price + " TL");
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 4
Price: 0 TL
```

RUN 2

```
> 12
Price: 10 TL
```

RUN 3

```
> 30
Price: 25 TL
```

RUN 4

```
> 70
Price: 15 TL
```

HINT

Assign the price in each branch, then print once at the end. Only upper bounds are needed when you test from the youngest group up.

Q6. Read a year and say whether it is a leap year (divisible by 4, except centuries, which must be divisible by 400).

JAVA SOLUTION

Q6.java

```java
import java.util.Scanner;

public class Q6 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int year = in.nextInt();
    boolean leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    if (leap) {
      System.out.println(year + " is a leap year");
    } else {
      System.out.println(year + " is not a leap year");
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 1900
1900 is not a leap year
```

RUN 2

```
> 2000
2000 is a leap year
```

RUN 3

```
> 2023
2023 is not a leap year
```

RUN 4

```
> 2024
2024 is a leap year
```

HINT

Store the whole condition in a boolean variable first; the if then reads like English. Keep the parentheses around (a && b).

Q7. Read three side lengths. Print "Not a triangle" if they cannot form one; otherwise classify it as Equilateral, Isosceles or Scalene.

JAVA SOLUTION

Q7.java

```java
import java.util.Scanner;

public class Q7 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int a = in.nextInt();
    int b = in.nextInt();
    int c = in.nextInt();
    if (a + b <= c || a + c <= b || b + c <= a) {
      System.out.println("Not a triangle");
    } else if (a == b && b == c) {
      System.out.println("Equilateral");
    } else if (a == b || b == c || a == c) {
      System.out.println("Isosceles");
    } else {
      System.out.println("Scalene");
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 3 4 5
Scalene
```

RUN 2

```
> 5 5 5
Equilateral
```

RUN 3

```
> 5 5 8
Isosceles
```

RUN 4

```
> 1 2 5
Not a triangle
```

HINT

Invalid when any pair sums to the third side or less: use ||. Then test equilateral BEFORE isosceles, because an equilateral triangle also has two equal sides.

Q8. Read a user name and a password (two words). The only valid login is admin / 1234. Print "Welcome, admin", "Wrong password" (right user, wrong password) or "Unknown user".

JAVA SOLUTION

Q8.java

```
import java.util.Scanner;

public class Q8 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    String user = in.next();
    String pass = in.next();
    if (user.equals("admin") && pass.equals("1234")) {
      System.out.println("Welcome, admin");
    } else if (user.equals("admin")) {
      System.out.println("Wrong password");
    } else {
      System.out.println("Unknown user");
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> admin 1234
Welcome, admin
```

RUN 2

```
> admin 0000
Wrong password
```

RUN 3

```
> bob 1234
Unknown user
```

HINT

Strings are compared with .equals(), never ==. Check the user name first, then the password.

Q9. Complete the truth table for (a || b) && !c by hand, then check yourself against the program below, which prints all eight rows.

JAVA SOLUTION

Q9.java

```
public class Q9 {
  public static void main(String[] args) {
    boolean a, b, c;
    System.out.println("a      b      c      (a || b) && !c");
    a = false; b = false; c = false;
    System.out.printf("%-6b %-6b %-6b %b%n", a, b, c, (a || b) && !c);
    a = false; b = false; c = true;
    System.out.printf("%-6b %-6b %-6b %b%n", a, b, c, (a || b) && !c);
    a = false; b = true; c = false;
    System.out.printf("%-6b %-6b %-6b %b%n", a, b, c, (a || b) && !c);
    a = false; b = true; c = true;
    System.out.printf("%-6b %-6b %-6b %b%n", a, b, c, (a || b) && !c);
    a = true; b = false; c = false;
    System.out.printf("%-6b %-6b %-6b %b%n", a, b, c, (a || b) && !c);
    a = true; b = false; c = true;
    System.out.printf("%-6b %-6b %-6b %b%n", a, b, c, (a || b) && !c);
    a = true; b = true; c = false;
    System.out.printf("%-6b %-6b %-6b %b%n", a, b, c, (a || b) && !c);
    a = true; b = true; c = true;
    System.out.printf("%-6b %-6b %-6b %b%n", a, b, c, (a || b) && !c);
  }
}
```

ACTUAL OUTPUT

OUTPUT

```
a      b      c      (a || b) && !c
false  false  false  false
false  false  true   false
false  true   false  true
false  true   true   false
true   false  false  true
true   false  true   false
true   true   false  true
true   true   true   false
```

HINT

Work column by column: first (a || b), then !c, then combine them with &&. The result is true only when at least one of a, b is true AND c is false.

Q10. Read a number, an operator (+ - * /) and another number, and print the result with a switch. Dividing by zero prints "Cannot divide by zero"; any other symbol prints "Unknown operator".

JAVA SOLUTION

Q10.java

```java
import java.util.Scanner;

public class Q10 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    double a = in.nextDouble();
    char op = in.next().charAt(0);
    double b = in.nextDouble();
    switch (op) {
      case '+':
        System.out.println(a + b);
        break;
      case '-':
        System.out.println(a - b);
        break;
      case '*':
        System.out.println(a * b);
        break;
      case '/':
        if (b == 0) {
          System.out.println("Cannot divide by zero");
        } else {
          System.out.println(a / b);
        }
        break;
      default:
        System.out.println("Unknown operator");
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 6 * 7
42.0
```

RUN 2

```
> 7 / 2
3.5
```

RUN 3

```
> 5 / 0
Cannot divide by zero
```

RUN 4

```
> 5 ^ 2
Unknown operator
```

HINT

Read the operator with next().charAt(0). Don't forget break after every case, and put the zero check inside the '/' case.

Q11. Read a month number (1–12) and a year; print how many days that month has. February depends on the leap year; an invalid month prints "Invalid month".

JAVA SOLUTION

Q11.java

```java
import java.util.Scanner;

public class Q11 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int month = in.nextInt();
    int year = in.nextInt();
    int days;
    switch (month) {
      case 4: case 6: case 9: case 11:
        days = 30;
        break;
      case 2:
        boolean leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
        days = leap ? 29 : 28;
        break;
      case 1: case 3: case 5: case 7: case 8: case 10: case 12:
        days = 31;
        break;
      default:
        days = -1;
    }
    if (days == -1) {
      System.out.println("Invalid month");
    } else {
      System.out.println(days + " days");
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 2 2024
29 days
```

RUN 2

```
> 2 2023
28 days
```

RUN 3

```
> 4 2024
30 days
```

RUN 4

```
> 13 2024
Invalid month
```

HINT

Group the 30-day and 31-day months with stacked case labels. Inside case 2, reuse the leap-year test and choose 29 or 28 with ?:.

Q12. Read two integers. Using the ternary operator, print the larger one and whether the two numbers are equal or different.

JAVA SOLUTION

Q12.java

```java
import java.util.Scanner;

public class Q12 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int a = in.nextInt();
    int b = in.nextInt();
    int larger = (a > b) ? a : b;
    String relation = (a == b) ? "equal" : "different";
    System.out.println("larger = " + larger + ", the numbers are " + relation);
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 5 9
larger = 9, the numbers are different
```

RUN 2

```
> -4 -4
larger = -4, the numbers are equal
```

RUN 3

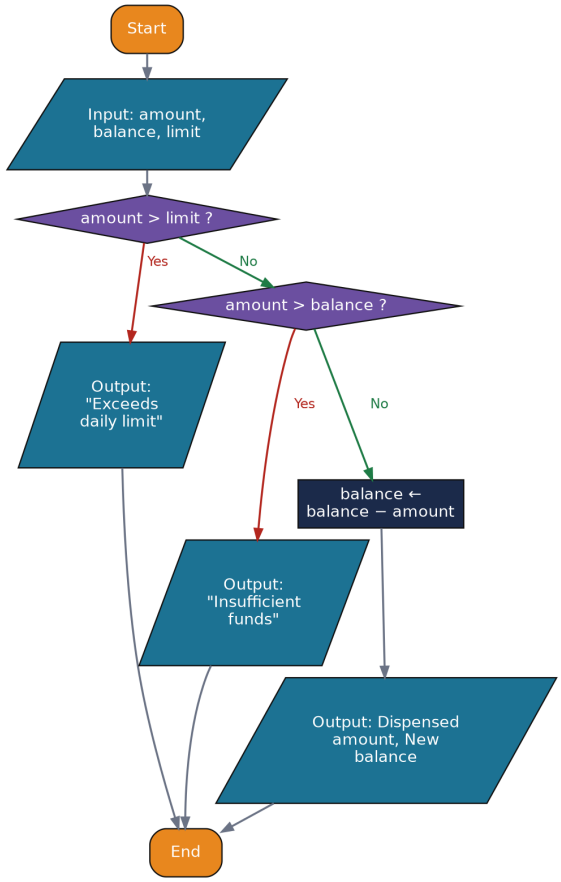
```
> 12 3
larger = 12, the numbers are different
```

HINT

int larger = (a > b) ? a : b; and a second ?: picks the word. Both ?: expressions produce a value, so no if-else is needed.

Q13. Translate the flowchart on the left into Java. It is the ATM from Week 2: read the amount, the balance and the daily limit.

FLOWCHART (the question)



JAVA SOLUTION

Q13.java

```java
import java.util.Scanner;

public class Q13 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int amount = in.nextInt();
    int balance = in.nextInt();
    int limit = in.nextInt();
    if (amount > limit) {
      System.out.println("Exceeds daily limit");
    } else if (amount > balance) {
      System.out.println("Insufficient funds");
    } else {
      balance = balance - amount;
      System.out.println("Dispensed " + amount);
      System.out.println("New balance: " + balance);
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> 500 1200 1000
Dispensed 500
New balance: 700
```

RUN 2

```
> 1500 3000 1000
Exceeds daily limit
```

RUN 3

```
> 900 400 1000
Insufficient funds
```

HINT

Two rejections come first (limit, then balance); the success branch updates the balance. An else-if ladder with a final else matches the diagram exactly.

Q14. Rock-paper-scissors: read two moves (r, p or s). Print "Draw", "Player 1 wins" or "Player 2 wins". Any other letter prints "Invalid move".

JAVA SOLUTION

Q14.java

```java
import java.util.Scanner;

public class Q14 {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    char p1 = in.next().charAt(0);
    char p2 = in.next().charAt(0);
    boolean ok1 = p1 == 'r' || p1 == 'p' || p1 == 's';
    boolean ok2 = p2 == 'r' || p2 == 'p' || p2 == 's';
    if (!ok1 || !ok2) {
      System.out.println("Invalid move");
    } else if (p1 == p2) {
      System.out.println("Draw");
    } else if ((p1 == 'r' && p2 == 's') || (p1 == 's' && p2 == 'p')
      || (p1 == 'p' && p2 == 'r')) {
      System.out.println("Player 1 wins");
    } else {
      System.out.println("Player 2 wins");
    }
  }
}
```

SAMPLE RUNS (typed input is marked with >)

RUN 1

```
> r s
Player 1 wins
```

RUN 2

```
> p p
Draw
```

RUN 3

```
> s p
Player 1 wins
```

RUN 4

```
> p s
Player 2 wins
```

RUN 5

```
> x r
Invalid move
```

HINT

Validate both moves first. Rock beats scissors, scissors beats paper, paper beats rock: list Player 1's three winning pairs with && and join them with ||.

Q15. Predict what this program prints, line by line, before you run it.

PROGRAM

Q15.java

```java
public class Q15 {
  public static void main(String[] args) {
    int x = 5, y = 0;
    if (x > 3 || y++ > 0) {
      System.out.println("A");
    }
    System.out.println(y);
    if (x > 10 && y++ > 0) {
      System.out.println("B");
    }
    System.out.println(y);
    if (x > 3 && y++ == 0) {
      System.out.println("C");
    }
    System.out.println(y);
  }
}
```

ACTUAL OUTPUT

OUTPUT

```
A
0
0
C
1
```

WHY

statement	prints	why
`if (x > 3 || y++ > 0)`	`A`	x > 3 is true, so y++ is never evaluated: y stays 0
`println(y)`	`0`	y was not touched
`if (x > 10 && y++ > 0)`	`(nothing)`	x > 10 is false, so && stops: y++ never runs
`println(y)`	`0`	still 0
`if (x > 3 && y++ == 0)`	`C`	x > 3 is true, so y++ runs: 0 == 0 is true, then y becomes 1
`println(y)`	`1`	the increment from the previous line

Wrap-up check (5 min)

Before leaving, each student should be able to answer out loud, pointing at their own code:
• “Why does Q2 use two separate ifs instead of one else-if ladder?”
• “In Q15, why does y stay 0 after the first if?”
• “Why .equals() in Q8 and not ==?”

Instructor notes

- Most common errors this week: = instead of ==, a stray semicolon after if ( ), missing braces around two statements, == on Strings, a forgotten break in switch, and else-if tests written in the wrong order. Have students read compiler messages top-down.
- Q1–Q3 should be quick; use them to check that everyone can compile and run. Q4 and Q5 are about ladder order: ask what changes if the tests are reversed.
- Q6–Q8 introduce && and ||. For Q9, insist on filling the truth table by hand before running the program.
- Q10–Q12 cover switch and ?:. Q11 is the first question that needs both a switch and a boolean expression — expect it to take longer.
- Q13 is the Week 2 ATM flowchart in Java; students who did the Week 2 lab will recognise it. Q14 is the capstone: validation, && and || together.
- Q15 is a paper exercise on short-circuit evaluation: a side effect (y++) inside a condition may or may not run. Ask for the written prediction first.
- Next lecture: while, for and do-while — the loop-back arrow from Weeks 1–2, in Java.